TIMING ANALYSIS AND OPTIMIZATION OF SEQUENTIAL CIRCUITS

TIMING ANALYSIS AND OPTIMIZATION OF SEQUENTIAL CIRCUITS

Naresh Maheshwari
Iowa State University

Sachin S. Sapatnekar
University of Minnesota



SPRINGER SCIENCE+BUSINESS MEDIA, LLC

Library of Congress Cataloging-in-Publication Data

Maheshwari, Naresh, 1970-
Timing analysis and optimization of sequential circuits / Naresh Maheshwari, Sachin S. Sapatnekar.
p. cm.
Includes bibliographical references and index.
ISBN 978-1-4613-7579-1 ISBN 978-1-4615-5637-4 (eBook)
DOI 10.1007/978-1-4615-5637-4
1. Integrated circuits--Very large scale integration--Design and construction--Data processing. 2. Computer-aided design. 3. Time-series analysis--Data processing.
I. Sapatnekar, Sachin S., 1967- . II. Title.
TK7874.75.M35 1998
621.39'5--dc21 98-42471
CIP

Copyright © 1999 by Springer Science+Business Media New York
Second printing 2001.
Originally published by Kluwer Academic Publishers in 1999
Softcover reprint of the hardcover 1st edition 1999
All rights reserved. No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, mechanical, photo-copying, recording, or otherwise, without the prior written permission of the publisher, Springer Science + Business Media, LLC.

Printed on acid-free paper.

Contents

List of Figures ix
Preface xiii

1. INTRODUCTION 1
   1.1 Performance Optimization of VLSI Circuits 1
   1.2 Outline of the Book 5

2. TIMING ANALYSIS OF SEQUENTIAL CIRCUITS 7
   2.1 Introduction 7
   2.2 Combinational Delay Modeling 10
   2.3 Clocking Disciplines: Edge-Triggered Circuits 17
   2.4 Resolving Short Path Violations 18
   2.5 Clocking Disciplines: Level-clocked Circuits 20
   2.6 Clock Schedule Optimization for Level-Clocked Circuits 26
   2.7 Timing Analysis of Domino Logic 27
   2.8 Concluding Remarks 31

3. CLOCK SKEW OPTIMIZATION 33
   3.1 The Notion of Deliberate Clock Skew 33
   3.2 Is Clock Skew Optimization Safe? 35
   3.3 Clock Tree Construction 35
   3.4 Clock Skew Optimization 40
   3.5 Clock Skew Optimization with Transistor Sizing 46
   3.6 Wave Pipelining Issues 60
   3.7 Deliberate Skews for Peak Current Reduction 62
   3.8 Conclusion 63

4. THE BASICS OF RETIMING 65
   4.1 Introduction to Retiming 65
   4.2 A Broad Overview of Research on Retiming 69
   4.3 Modeling and Assumptions for Retiming 72
   4.4 Minperiod Optimization of Edge-triggered Circuits 74

4.5 Level-clocked Circuits 88
4.6 Concluding Remarks 97

5. MINAREA RETIMING 99
5.1 The Leiserson-Saxe Approach 100
5.2 The Minaret Algorithm 105
5.3 Minarea Retiming of Level-Clocked Circuits 113

6. RETIMING CONTROL LOGIC 123
6.1 Minperiod Initial State Retiming via the State Transition Graph 124
6.2 Minperiod Initial State Retiming via Reverse Retiming 127
6.3 Minarea Initial State Retiming 129
6.4 Maintaining Initial States With Explicit Reset Circuitry 144

7. MISCELLANEOUS ISSUES IN RETIMING 147
7.1 Retiming and Testing 148
7.2 Verification Issues 154
7.3 Retiming for Low Power 156
7.4 Retiming with Logic Synthesis 157
7.5 Retiming for FPGA's 161
7.6 Practical Issues 162
7.7 Conclusion 168

8. CONCLUSION 169

References 171

4 THE BASICS OF RETIMING

4.1 INTRODUCTION TO RETIMING

Retiming is a powerful sequential circuit optimization technique for improving the performance of sequential circuits. The concept of retiming is the notion of moving storage devices across memoryless computational elements to improve the performance without changing the input-output latency. Although retiming can operate on gate level netlists, or on higher-level abstractions such as data flow graphs, communication graphs, and processor schedules, our treatment will focus on circuit-level optimizations.

At the circuit level, the storage devices, or registers, may either be edge-triggered flip-flops or level-sensitive latches, commonly referred to as FF's or latches, respectively. The computational nodes at the circuit level are typically combinational gates. Retiming moves registers across gates without changing the number of registers in any cycle or on any path from the primary inputs to the primary outputs, thereby preserving the input-output latency of the circuit. Since retiming does not directly affect the combinational part of the circuit, the circuit behavior remains unchanged. However, since retiming can change the boundaries of combinational logic, it has the potential to affect the results of combinational synthesis techniques that are applied to the sequential circuit.

This chapter begins with an overview of the retiming procedure. Next, the assumptions and models common to many retiming algorithms are presented, followed by a description of the Leiserson-Saxe theory for retiming

N. Maheshwari et al., *Timing Analysis and Optimization of Sequential Circuits*
© Kluwer Academic Publishers 1999

edge-triggered circuits is presented. Finally, we discuss fast and efficient methods for retiming large edge-triggered and level-clocked circuits to minimize their clock period.

4.1.1 *Types of Retiming*

Retiming may be performed to improve the circuit behavior with respect to several possible objective functions, some of which are outlined below.

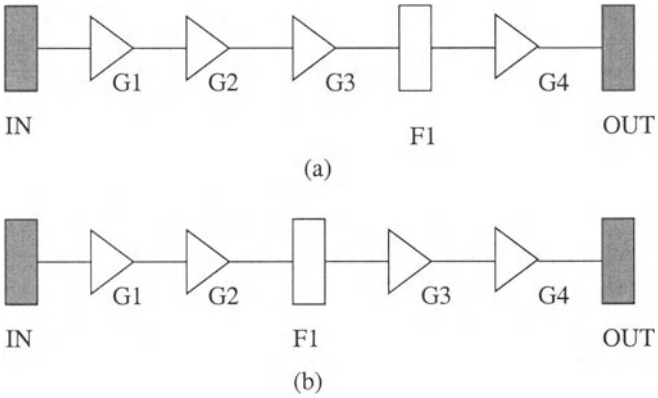


Figure 4.1. Effect of retiming on clock period

Clock Period Algorithmically, the simplest objective function used in retiming is the minimization of the clock period. Since the clock period in a circuit with edge-triggered FF's is given by the maximum combinational stage delay, the FF's may be relocated to reduce the clock period. For the circuit shown in Figure 4.1(a), with unit delay gates and edge-triggered FF's, the clock period is 3 time units. If we relocate register F1 from the output of gate G3 to its input, we obtain the circuit in Figure 4.1(b), with a clock period of 2 units. Notice that the input-output behavior is left unchanged by retiming since the output is produced after two clock cycles in both the original and the retimed circuit. Thus, relocating registers can reduce the clock period of a circuit, and retiming can be used to relocate registers with the objective of minimizing the clock period. A retiming that minimizes the clock period of a circuit is termed a *minperiod retiming*. Retiming a circuit to achieve a specified target clock period is a special case of minperiod retiming, and is often called specified-period retiming.

Area Since retiming does not affect the combinational part of the circuit, the area overhead of the combinational logic remains constant under retiming. The method may, however, affect the overall area of the circuit since it may alter the number of registers in the circuit.

Two retimed versions of the same circuit could have the same input-output behavior and clock period, but could use a different number of registers, as

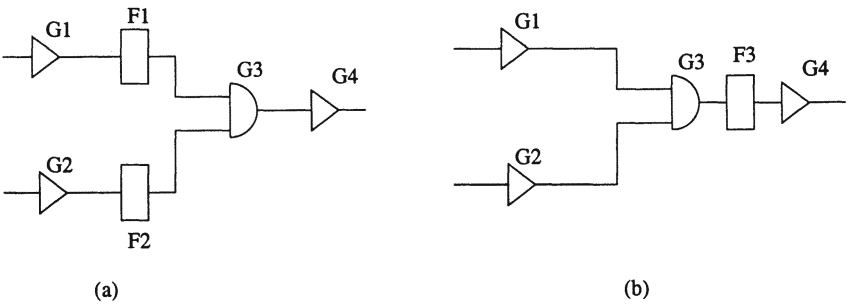


Figure 4.2. Effect of retiming on number of registers

illustrated in Figure 4.2. The circuit in Figure 4.2(a) requires two registers, while that in Figure 4.2(b) requires only one register.

Therefore, one could apply retiming with the object of minimizing the number of registers in the circuit, while leaving the input-output latency unchanged. This may be done without any constraint on the clock period of the resulting circuit, or subject to a target clock period. The former is called unconstrained minarea retiming while the latter is referred to as constrained minarea retiming or simply *minarea retiming*. In practice, minarea retiming is a more useful form of the retiming transformation than minperiod retiming.

Power The power dissipated in a circuit depends on the product of the switching activity and the load capacitance at the output of a gate, summed over all gates. Since registers can filter out glitches, altering their locations can affect the switching activity at gate outputs; moreover, it can also alter the load capacitance seen by the gates. Thus, retiming can change the power dissipation of a circuit, and an appropriately chosen retiming may be used to optimize the power by placing registers at nodes with high switching activity values and high capacitive loads.

Testability The relocation of registers can change the state encoding in sequential circuits, thus affecting the test generation time and the number of redundant faults. The repositioning of registers also affects the length of the scan chains required for partial or full scan designs. Retiming can, therefore, be used to improve the testability of sequential circuits.

Quality of Logic Optimization Most logic optimization techniques operate on combinational logic blocks separated by register boundaries. Hence, changing these register boundaries by retiming the registers affects the qualityof results obtained by logic optimization.

A brief survey of publications describing these research activities is presented in Section 4.2, and in the next few chapters, we will describe these methods in varying degrees of detail.

4.1.2 Types of Circuits

Algorithms for retiming a circuit must address the specific requirements of a circuit, and the clocking discipline used. Four major classes of circuits are described below.

Edge-triggered circuits In an edge-triggered circuit, the clock period is given simply by the largest combinational delay. The first publications on retiming concentrated on handling this class of circuits.

Level-clocked circuits Level-sensitive latches are, by definition, transparent during the period when the clock signal is active; this transparent nature gives level-clocked circuits the potential to operate at a faster clock period[1]. These circuits require less area than their edge-triggered counterparts, not only because the cycle stealing reduces the amount of transistor sizing required to meet the timing goals, but also because individual latches require a smaller amount of area than edge-triggered FF's. Unfortunately, the analysis of level-clocked circuits is more complicated than edge-triggered circuits, and hence, algorithms for finding an optimal retiming can be computationally expensive.

Control Logic Control logic involves an implementation of finite state machines (FSM's), and hence, the registers in control circuitry are associated with the FSM states. Retiming alters the locations of these registers, and consequently, the state encoding of the FSM. Thus, issues regarding safe replaceability become important. In circuits that have a meaningful initial state, it is important to find a retimed circuit with an equivalent initial state; many retimings of a circuit that are otherwise valid may not have equivalent initial states. To see this, consider the circuit in Figure 4.3(a). If we wish to move FF's A and B across gate G1 (to FF C in Figure 4.3(b)), we must find an initial value of FF C that is equivalent to their initial values. If FF's A and B have conflicting values, no such equivalent initial value exists at FF C. Thus, additional constraints must be imposed to ensure the presence of an equivalent initial state while retiming control logic.

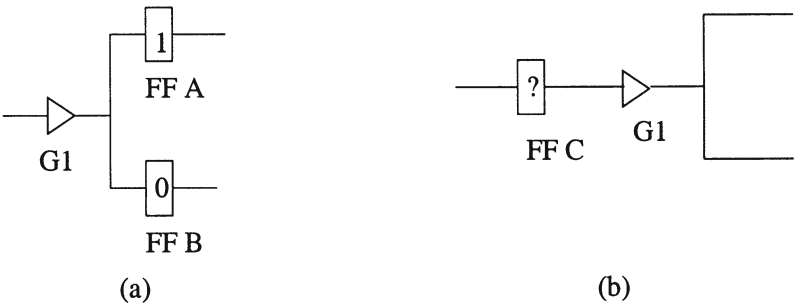


Figure 4.3. Equivalent initial states in reverse retiming.

FPGA's Field Programmable Gate Arrays (FPGA's) present requirements that are different from those in conventional combinational logic. For example, in LUT-based FPGA's the amount of logic is dependent on the number of inputs, and not on the complexity of the logic. Further, since FPGA's have limited resources with memory elements at fixed locations, extra constraints are placed on the movement of memory elements during retiming. The issue of combined synthesis with retiming shows the maximal gains in FPGA optimization.

4.2 A BROAD OVERVIEW OF RESEARCH ON RETIMING

Since retiming was introduced by Leiserson and Saxe [LRS83, LS91], a significant amount of research has been carried out, both in academia [Sax85, Pap90, Cha93, Pap93, Ran93, She93, Deo94, Eve94, Lal96, Sin96, Has97, Ran97] and in industry (for example, at IBM, Philips and Synopsys). In this section, we present a brief literature survey of retiming-related research. For a good introduction and overview of retiming, the reader is also referred to [She97] and to Section 9.3.1 of [De 94]. In the remainder of this book, more detailed descriptions of these issues will be presented.

Edge-triggered Circuits Leiserson and Saxe introduced algorithms for minperiod and minarea retiming of edge-triggered circuits [LRS83, LS91]. The basis for this work is a graph-based representation of the circuit, and a procedure used to map the retiming problem on to polynomial time graph algorithms. This approach will be referred to in this book as the "Leiserson-Saxe approach." circuit is represented by a graph and polynomial time algorithms are presented. The major contribution of this work is in formulating the theory of retiming, rather than in presenting experimental implementations. The minperiod retiming problem is solved by performing a binary search for the best clock period, checking the feasibility of the clock period using a Bellman-Ford-like relaxation algorithm. The minarea problem is formulated as a linear program (LP), which is shown to be the dual of a mincost network flow problem.

Shenoy and Rudell presented an efficient and clever implementation of the Leiserson-Saxe algorithms in [SR94]. Their main contributions include reducing the memory requirements from $O(|G|^2)$ to $O(|G|)$, where $|G|$ is the number of gates in the circuit, and the use of back-pointers to speed up the feasibility check during the binary search for minimum clock period. At about the same time as [SR94], a technique for reducing the number of constraints in the minarea LP was presented in [vVA+94].

The ASTRA algorithm [DS95, SD96] exploited the retiming-skew equivalence for fast minperiod retiming. ASTRA first finds a minimum period achievable by skew optimization, and then translates these skews into retiming. Circuits with 20,000 gates were shown to be retimed in under two minutes.

The Minaret algorithm [MS97a, MS98b] modifies ASTRA to efficiently obtain bounds on the retiming variables, and uses them to significantly reduce the size of the minarea LP. This enables Minaret to perform minarea retiming on circuits with 50,000 gates in under 15 minutes.

Level-Clocked Circuits A signal that flows through a level-sensitive latch during its transparent phase can initiate the computation of the next combinational stage before the beginning of the next clock cycle; this phenomenon is called cycle stealing. Due to cycle stealing, level-clocked circuits have the potential to operate faster, and require less area. Algorithms to retime single phase level-clocked circuits are presented in [SBS91], and techniques based on the Leiserson-Saxe model for retiming multi-phase level-clocked circuits were presented in [ILP92, LE92, LE94]. TIM [PR93b] is a comprehensive timing analysis and optimization CAD tool for level-clocked circuits that is available in public domain, and has been used to empirically compare the performance of edge-triggered and level-sensitive circuits in [PR93a]. The work in [IP96] presented a polynomial-time algorithm for pipelining two-phase, level-clocked circuits under a bounded delay model.

The ASTRA and Minaret algorithms for edge-triggered circuits have been extended to level-clocked circuits in [MS96, MS98a] respectively. This enables minperiod and minarea retiming of level-clocked circuits with tens of thousands of gates in very reasonable time.

Retiming with Equivalent Initial States Traditional retiming algorithms do not pay any regard to initial states or power-on states of circuits, and are not very useful for control logic. Control logic usually has meaningful initial states, and any useful retiming must also find a new initial state for the transformed circuit that is equivalent to the initial state of the original circuit.

A method for minperiod retiming with equivalent initial states was presented in [TB93], using only the so-called forward retiming moves. In some cases, this approach may require modifications in the combinational part of the circuit. An efficient technique for performing these modifications is presented in [SMB96]. An alternative approach, termed reversed retiming [SSE95, ESS96], uses a minimum number of reverse (backward) retiming moves, and precludes the need for any modifications to the combinational part of the circuit. An approach to minimum area retiming that maintains initial states is presented in [MS97b].

Low Power Retiming can affect the power dissipation of a circuit since it alters the amount of switching in a circuit, as also the fanout capacitance driven by various elements in the circuit. A mechanism for reducing power by retiming was presented in [MDG93]; it places FF's on interconnects with high switching activity. The approach in [LP96] presented algorithms to reduce power by retiming only one phase in a two-phase circuit. The advantage of

retiming only one phase is that it preserves the testability of the circuit. A similar approach is taken in [SSWN97] to reduce power in DSP designs.

Testing and Verification Retiming can be used both to improve the testability of a circuit, and as an aid to automatic test generation. In the former case, circuit is actually implemented in the retimed form, while in the latter case the retimed circuit is merely used by the test generator to generate test vectors, but the original circuit is implemented. Various researchers have worked on characterizing the effects of FF relocation on the redundancy of faults [DC94, YKK95a, YKK95b, DB96], and the corresponding effects on ATPG run time [MEMR96]. In [EMRM95, EMRM97], it was shown that retiming preserves testability with respect to a single stuck-at-fault test set by adding a prefix sequence of a pre-determined number of arbitrary input vectors. Retiming has been used to improve testability [DC94] by attempting to convert sequential redundancies to combinational redundancies. Retiming may also be employed for reducing test lengths in scan-based designs [KT94, HKK95, HKK96], and for improving pseudo-exhaustive built-in self test [KTB93, LKA95a, LKA95b, LKA95c].

The work in [SPRB95] shows that while an accurate logic simulation may distinguish a retimed circuit from the original circuit, a conservative three-valued simulator cannot do so. Techniques for verification of retimed circuits are presented in [HCC96, HCC97, RSSB98].

Enhancements to Retiming The traditional retiming approaches assume the gate delays to be fixed, and all FF delays to be equal. Since these are only approximations, much effort has been spent in incorporating improved delay models into retiming. Delay models that incorporate clock skews, register delays, etc., are presented in [SFM93, SF94, LP95a, LP95b, SFM97], while [KO95] presents retiming under a variable delay model.

Retiming has also been extended to handle multiple clocks and registers with enable ports in [Mar96, LVW97], and to handle gated clocks and precharged structures [Ish93]. Various efforts have been made to combine retiming with other logic synthesis techniques, e.g., [MSBS90, MSBS91, PR91, Lin93, LKW93, BC96, Pan97, AB98]. Architectural retiming [Has97] modifies the combinational part of a circuit to increase the number of registers on a critical cycle or path without increasing the perceived latency.

Other Applications Retiming has been used in numerous other applications: during the technology mapping step in FPGA synthesis [TSS92, WR93, CW94, PL95, CW96], to improve circuit partitioning [LSC+93], for scheduling in high level synthesis [CS93a, WBL+94], in multiprocessor scheduling [CS92], for system level throughput optimization include [MH91, DPP95], etc. Retiming has also been used extensively in DSP applications [Dun92, PR92, Geb93, PS96, PSB96].

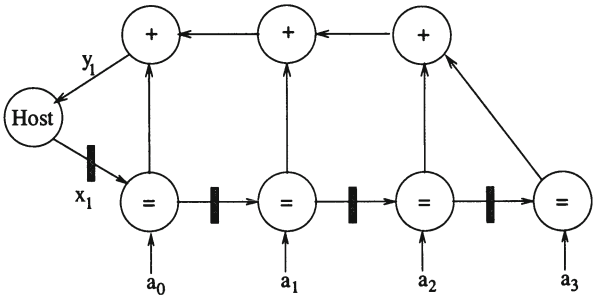


Figure 4.4. A correlator circuit.

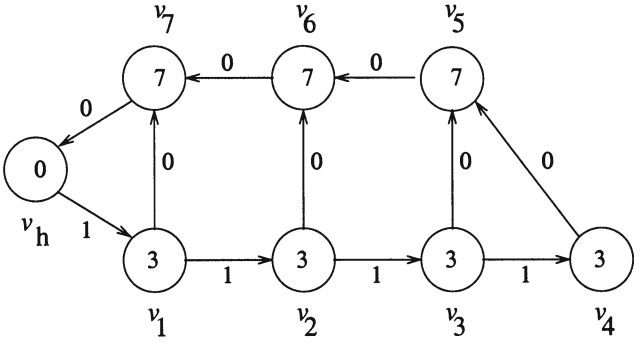


Figure 4.5. Graph model of the correlator example.

4.3 MODELING AND ASSUMPTIONS FOR RETIMING

Any detailed presentation of material on retiming necessarily requires a section explaining the models and preliminaries that are used by most algorithms. In this section, we will present a graph-theoretic foundation that will be utilized later in our exposition on retiming.

In the ensuing discussion, we will consider the example of a digital correlator circuit from [LS91], shown in Figure 4.4, to motivate the procedure of retiming by demonstrating the performance improvements on this example. The design consists of two types of functional elements: adders (denoted by a '+' symbol) and comparators (denoted by a '=' symbol). The boxes between the comparators are registers that act to shift the data. The delay of an adder is 7 units and that of a comparator is 3 units. Although this design is functionally correct, it has poor timing characteristics.

For the purposes of applying a retiming algorithm, it is useful to abstract the problem from the circuit level into a directed graph representation, $G(V, E, d, w)$, where V is the set of vertices and E the set of edges. Each vertex $v \in V$ models a functional element with a fixed propagation delay, $d(v)$, that remains unaltered during the retiming process. A special vertex called the *host* is introduced in the graph, with outgoing edges connected to all primary inputs of the circuit, and incoming edges from all primary outputs to the host vertex. The host ver-

tex is assigned a propagation delay of zero, i.e., $d(host) = 0$. The directed edges model the interconnections between functional elements. Each edge $e_{uv} \in E$ connects an output of functional element u to an input of a functional element v, with a weight of $w(e_{uv})$ that corresponds to the number of registers between u and v. The graph corresponding to the correlator circuit in Figure 4.4 is shown in Figure 4.5.

As an informal definition, a retiming move in a circuit is caused by moving all of the memory elements at the input to a combinational block to all of its outputs, or vice versa. A sequence of such movements is referred to as a retiming of the circuit.

Equivalently, working on the graph representation of the circuit, a retiming is an integer-valued labeling of the vertices, $r : V \rightarrow \mathbf{Z}$, where $\mathbf{Z}$ is the set of integers. The weight of an edge e_{uv} after retiming, denoted by $w_r(e_{uv})$, is given by

$$w_r(e_{uv}) = r(v) + w(e_{uv}) - r(u) \tag{4.1}$$

The retiming label, $r(v)$, for a vertex v represents the number of register stages that have been moved from its outputs to its inputs. Retiming can, therefore, also be viewed as an assignment of a lag value, $r(v)$, to every vertex v in the circuit. A path originating at vertex u and terminating at vertex v is represented as $u \rightsquigarrow v$. For a path $p : u \rightsquigarrow v$, one may define its weight, $w(p)$, as the sum of the weights on the edges on p, and its delay, $d(p)$, as the sum of the weights of the vertices on p. A path with $w(p) = 0$ corresponds to a purely combinational path with no registers on it. Therefore, the clock period, P, can be calculated as

$$P = \max_{\forall\, p | w(p)=0} \{d(p)\} \tag{4.2}$$

The Leiserson-Saxe method presents a systematic technique for computation of the retiming labels, where the problem is formulated as a mixed integer linear program (MILP). An important concept used in the approach is the notion of the W and D matrices. The matrices are defined for all pairs of vertices (u, v) such that there exists a path, $p : u \rightsquigarrow v$, that does not include the host vertex. The formal definition of the matrices is as follows:

$$W(u, v) = \min_{\forall\, p: u \rightsquigarrow v} \{w(p)\} \tag{4.3}$$

$$D(u, v) = \max_{\forall\, p: u \rightsquigarrow v \text{ and } w(p) = W(u,v)} \{d(p)\} \tag{4.4}$$

In plain English, $W(u, v)$ denotes the minimum latency, in clock cycles, for the data flowing from u to v, and $D(u, v)$ gives the maximum delay from u to v over all paths with that minimum latency. The (W, D) pairs are used to generate constraints, and the essential reason why $D(u, v)$ is important is that if all memory elements between u and v are removed during retiming, then the delay between the two vertices on the path corresponding to $D(u, v)$ will be the maximum purely combinational delay between the two vertices. In the next section, we will use this information to ensure that if $D(u, v) > P$, the clock

period, then the value of $W(u,v)$ after retiming is at least 1. The important point here is that the delays on all but the minimum weight path are irrelevant.

The W and D matrices can easily be computed as follows. Each edge, e_{uv}, is reweighted with the ordered pair $(w(e_{uv}), -d(u))$, and the all-pairs shortest paths are computed. Any standard procedure for finding all-pairs shortest paths, e.g., the Floyd-Warshall method, or Johnson's algorithm [CLR90], may be used, with the following change that allows an ordered pair of weights to be correctly interpreted. During the relaxation process, comparisons between ordered pairs are performed in lexicographic order, i.e., amongst two pairs, $p_1 = (a_1, b_1)$ and $p_2 = (a_2, b_2)$, we say that $p_1 < p_2$ if $(a_1 < a_2)$ or $(a_1 = a_2$ and $b_1 < b_2)$. At the end of this computation, if the shortest-path weight between two vertices u and v is (x, y), then $W(u,v) = x$ and $D(u,v) = d(v) - y$.[2]

4.4 MINPERIOD OPTIMIZATION OF EDGE-TRIGGERED CIRCUITS

The algorithms presented in this section are all of polynomial time complexity. We will first present algorithms that are rooted in the techniques presented by Leiserson and Saxe [LS91] for retiming edge-triggered circuits, and follow these by discussing how they may be applied efficiently in practice. Finally, we will describe a method that applies the retiming-skew relationship to find an efficient minperiod retiming solution.

In each of these methods, the minimum period obtainable under retiming is calculated by performing a binary search over all possible clock periods. At each step in the binary search, an attempt is made to retime the circuit for the current value of the clock period. The smallest period for which retiming succeeds is returned as the best clock period.

4.4.1 Leiserson-Saxe-based Algorithms for Minperiod Retiming

Any retiming solution must satisfy the following two inequalities

$$\begin{aligned} r(u) - r(v) &\leq w(e_{uv}) \quad && \forall e_{uv} \in E \\ r(u) - r(v) &\leq W(u,v) - 1 \quad && \forall D(u,v) > P \end{aligned} \tag{4.5}$$

The first constraint ensures that the weight of each edge, e_{uv} (i.e., the number of registers between the output of gate u and the input of gate v), after retiming is nonnegative, i.e., $w_r(e_{uv}) \geq 0$. We will refer to these constraints as *circuit constraints*. The second constraint ensures that after retiming, each path whose delay is larger than the clock period has at least one register on it. These constraints, being dependent on the clock period, are often referred to as *period constraints*.

For a fixed clock period, the set of constraints (4.5) correspond to a system of difference constraints. This system can be represented by a constraint graph that may be solved by applying the Bellman-Ford method [CLR90]. If the system of constraints is infeasible, this will be indicated by the Bellman-Ford procedure. The smallest clock period for which the constraints are feasible is the solution to the minperiod problem.

This notion leads to the first algorithm, OPT1 [LS91]. As mentioned earlier, in the outer loop, a binary search is performed on the value of the clock period, and for each clock period, the Bellman-Ford algorithm is applied to check for feasibility of the constraint set. The binary search for the minimum value of P is justified by the fact that the feasible values of P form a continuous interval. This is due to the fact that the system (4.5) of inequalities is linear in P, making the projection of the solution space is convex in P, i.e., it is an interval. The OPT1 algorithm requires the calculation of the W and D matrices, and is expensive in terms of computation time ($O(V^3 \log V)$) and memory space ($O(V^2)$).

An alternative $O(|V||E|\log|V|)$-time algorithm from [LS91] is a more practical option for obtaining a retiming. The main routine FEAS is invoked during the binary search to check whether a specified clock period, P, is feasible or not. It proceeds by calling a subroutine, CP, that calculates the clock period of the circuit by systematically identifying the purely combinational path that has the largest delay. For any gate (or, in general, functional element) whose delay from an FF exceeds P, the retiming label, r, is incremented.

```
Algorithm FEAS
Given a synchronous circuit G = ⟨V, E, d, w⟩, and a desired clock
period P, return a retiming r of G such that the clock period
of the retimed circuit Φ(G_r) ≤ P.
{
  1. For each vertex v ∈ V, set r(v) ← 0.
  2. Repeat the following |V| − 1 times
     2.1 Compute the graph G_r with the existing values of r.
     2.2 Run Algorithm CP on the graph G_r to determine Δ(v)
         for each vertex v ∈ V.
     2.3 For each v such that Δ(v) > P, set r(v) ← r(v) + 1.
  3. Run Algorithm CP on the circuit G_r. If Φ(G_r) > P,
     then no feasible retiming exists. Otherwise, r is the
     desired retiming.
}

Algorithm CP
This algorithm computes the clock period Φ(G) for a synchronous
circuit G = ⟨V, E, d, w⟩.
{
  1. Let G_0 be the subgraph of G that contains precisely those
     edges e such that the register count w(e) = 0.
  2. Perform a topological sort on G_0, totally ordering
     its vertices so that if there is an edge from vertex u
     to vertex v in G_0, then u precedes v in the total order.
  3. Go through the vertices in the order defined by the
     topological sort.
     On visiting each vertex v, compute Δ(v) as follows:
```

a. If there is no incoming edge to v, $\Delta(v) \leftarrow d(v)$.
b. Otherwise, $\Delta(v) \leftarrow d(v) + \max\{\Delta(u) : u \xrightarrow{e} v \text{ and } e \in G_0\}$.

4. The clock period $\Phi(G)$ is $\max_{v \in V} \Delta(v)$.

}

The advantage of this method is that it does not require the explicit calculation of the W and D matrices, thereby reducing the memory overhead. Step 2 of FEAS can be shown to be equivalent to applying the Bellman-Ford algorithm on the graph G.

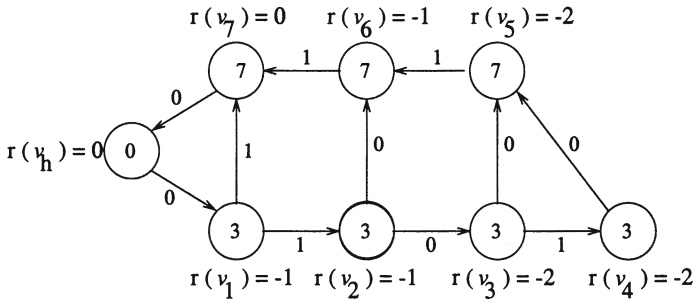


Figure 4.6. Retimed graph of the correlator.

The application of either of these retiming procedures may be used to assign the r labels to each vertex in the graph corresponding to the correlator circuit shown in Figure 4.4. For the unretimed circuit, the clock period is 24 units, which corresponds to the sum of the propagation delays along the longest register-free path, $v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_7$. The graph obtained on applying minperiod retiming to Figure 4.5 is shown in Figure 4.6. The r labels describe the manner in which flip-flops are added or removed from the original graph. For instance, the edge (v_1, v_7) has one more register since $r(v_7) - r(v_1) = 1$ (see relation (4.1)). Since retiming preserves the input-output latency of the circuit, the original and the retimed circuits corresponding to graphs in Figure 4.5 and Figure 4.6, respectively, must have the same input/output behavior. However, the computations performed by v_1 in Figure 4.5 lead, by one clock tick, the same computations performed by v_1 in Figure 4.6. This is the physical meaning of the fact that the vertex v_1 is assigned a lead of one clock tick or equivalently, a lag $r(v_1) = -1$. The clock period of the retimed circuit is 13 time units, corresponding to the sum of the propagation delays along the longest path, $v_2 \rightarrow v_3 \rightarrow v_5$.

Implementational Notes. An efficient implementation of the minperiod retiming algorithm was presented in [SR94], using a predecessor heuristic. The procedure is similar to the method of [Szy92], presented in Section 3.4.5, and its adaptation to this problem is presented here. The predecessor heuristic maintains a predecessor vertex pointer, denoted by `pred()`, for each vertex, which is initialized to the empty set. When $\Delta(v)$ is computed for each vertex

v, a reference to a vertex u is stored, where u satisfies the property that there exists a zero-weight path $p : u \rightsquigarrow v$ and $\Delta(v) = d(p)$. If $\Delta(v) > P$, then the procedure sets `pred(v) = u`.

Consider a cycle in the predecessor subgraph with vertices $u_1, \cdots, u_k = u_0$, i.e., `pred`$(u_i) = u_{i-1}$, $i = 1, \cdots, k$.[3] Let $p_{i-1,i}$ denote the path $u_{i-1} \rightsquigarrow u_i$ used in the computation of $\Delta(u_i)$. During each iteration in Step 2 of FEAS, the retiming labels increase by at most 1, and before the update, the weight on each path $w(p_{i-1,i}) = 0$. After the update,

$$w_r(p_{i-1,i}) = r(u_i) - r(u_{i-1}) + w(p_{i-1,i}) = r(u_i) - r(u_{i-1}) \leq 1$$

Therefore, $\sum_{i=1}^{k} w_r(p_{i-1,i}) \leq k$. Since $P < d(p_{i-1,i})$ for each i, we have

$$\begin{aligned} P \cdot k &< \sum_{i=1}^{k} d(p_{i-1,i}) \\ \Rightarrow P \cdot \sum_{i=1}^{k} w_r(p_{i-1,i}) &< \sum_{i=1}^{k} d(p_{i-1,i}) \end{aligned} \tag{4.6}$$

As long as the clock period is chosen so that this condition is true, a feasible retiming will not be possible. Therefore, it is essential to choose a clock period for which this condition is violated; this provides an updated lower bound for the binary search, given by

$$P \geq \frac{\sum_{i=1}^{k} d(p_{i-1,i})}{\sum_{i=1}^{k} w_r(p_{i-1,i})} \tag{4.7}$$

The use of this bound to update the lower bound of the binary search was shown in [SR94] to provide tremendous improvements in the execution time of the algorithm.

4.4.2 The ASTRA Approach

The Relationship Between Clock Skew and Retiming. The basis of the ASTRA approach [DS95, SD96] is the relationship between clock skew and retiming, as illustrated by the following example on a circuit with edge-triggered flip-flops. Let us first consider the use of intentional clock skews for improving the circuit performance. Consider the circuit in Figure 4.7, which has unit delay inverters and negative edge-triggered flip-flops. The delays of the first and the second combinational block are 3 units and 1, respectively. Therefore, the fastest allowable clock has a period of 3 units. If an attempt is made to operate the circuit with a clock period of 2 units, then we have a long path violation at FF1, because the required data arrival time at FF1 is 2 units (assuming zero setup time), but the data arrives at 3 units, as shown in Figure 4.7. However, if a skew of +1 unit is applied to the clock line to FF1, as shown in Figure 4.8, then the circuit can be operated without any long path violation, even for a clock period of two units[4].

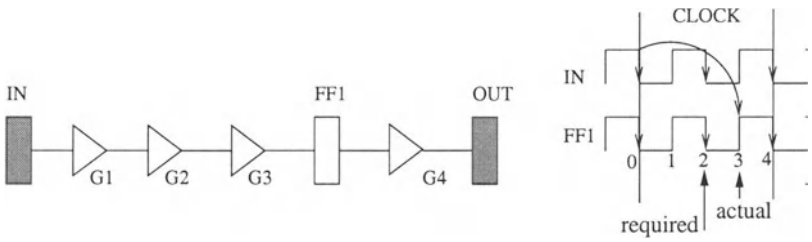


Figure 4.7. An example of a circuit with long path violation.

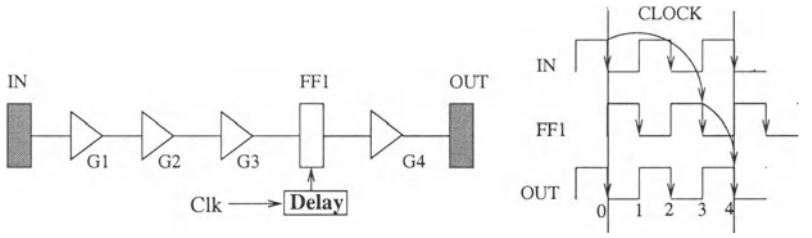


Figure 4.8. Using clock skew to correct the long path violation.

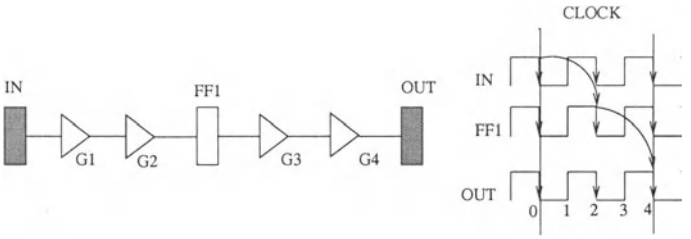


Figure 4.9. Retiming for clock period reduction.

It is easy to see that for the given circuit, the period can also be minimized to two units by retiming, if FF1 is moved to the left across the inverter G3, as shown in Figure 4.9.

In each case, one unit of time is borrowed by the first combinational block from the second; the manner in which cycle-borrowing occurs may either be via the vehicle of clock skew or via retiming. As a prelude to a more formal presentation of the relationship between clock skew and retiming, consider a flip-flop j in a circuit, as shown in Figure 4.10(a). As explained in Section 3.4.1, for every combinational path from a flip-flop i to j with delay $d(i,j)$, the following constraints must hold to prevent zero-clocking and double-clocking, respectively:

$$\begin{aligned} x_i + \overline{d}(i,j) + T_{setup} &\leq x_j + P \\ x_i + \underline{d}(i,j) &\geq x_j + T_{hold}, \end{aligned} \tag{4.8}$$

where x_i and x_j are the skews at flip-flops i and j, respectively. Similar constraints can be written for every combinational path from flip-flop j to k with minimum delay $\underline{d}(j,k)$ and maximum delay $\overline{d}(j,k)$.

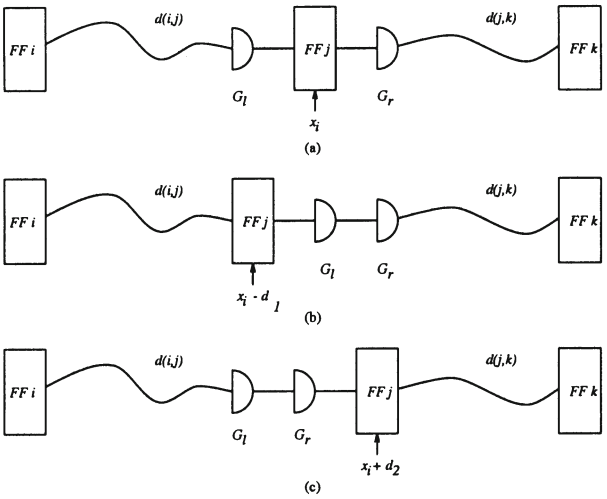


Figure 4.10. Relationship between retiming and skew.

Theorem 4.1 *For a circuit that operates at a clock period P, satisfying the double-clocking and zero-clocking delay constraints,*
(a) *retiming a flip-flop by moving it against the direction of signal propagation across a single-input, single-output gate of delay d_1 is equivalent to decreasing its skew by d_1.*
(b) *retiming a flip-flop by moving it in the direction of signal propagation across a single-input, single-output gate of delay d_2 is equivalent to increasing its skew by d_2.*

Proof:

(a) Consider the case where flip-flop j is moved to the left across the single-input gate G_l that has delay d_1. For a combinational path from an FF i to j, if x_j and x'_j are, respectively, the skews at flip-flop j before and after the relocation operation, then the following relationships must be satisfied:

$$\begin{aligned} x_i + [d(i,j) - d_1] + T_{setup} &\leq x'_j + P \\ x_i + [d(i,j) - d_1] &\geq x'_j + T_{hold} \end{aligned}$$

From relation (4.8), this implies that setting $x'_j = x_j - d_1$ achieves the same effect. Similarly, for a combinational path from j to a flip-flop k, it can be shown that setting $x'_j = x_j - d_1$ in the original circuit achieves the same effect. The proof to (b) is analogous and is omitted. □

A generalization of Theorem 4.1 to multi-input multi-output combinational blocks is as follows:

Theorem 4.2
(a) *Retiming transformations may be used to move flip-flops from all of the n inputs of any combinational block to all of its outputs. The equivalent skew of the relocated flip-flop at output j, considering long path constraints only, is given by*

$$x_j = \max_{1 \le i \le n} (x_i + \overline{d}(i,j)) \tag{4.9}$$

where the $x_i, 1 \le i \le n$, are the skews at the input flip-flops, x_j is the equivalent skew at output j, and $\overline{d}(i,j)$ is the worst-case delay of any path from i to j.
(b) *Similarly, flip-flops may be moved from all of the m outputs of any combinational block to all of its inputs, and the equivalent skew at input k, considering long path constraints only, is given by*

$$x_k = \min_{1 \le j \le m} (x_j - \overline{d}(k,j)) \tag{4.10}$$

where the $x_j, 1 \le j \le m$, are the skews at the input flip-flops, x_k is the equivalent skew at input k, and $\overline{d}(k,j)$ is the worst-case delay of any path from k to j.

The proof is omitted and proceeds along the lines of that for Theorem 4.1[5]. Theorem 4.2 implies that if one were to calculate the optimal clock skews, the corresponding retiming could be found by moving flip-flops with positive [negative] skews against [in] the direction of signal propagation until the skews at the flip-flops are nearly equal to zero. It must be noted that since gate delays take on discrete values, it is not possible to guarantee that the skew at a flip-flop can always be reduced to zero through retiming operations.

An alternative view of the same procedure is as follows. Retiming may be thought of as a sequence of movements of flip-flops across gates. Starting from the final retimed circuit, where all of the skews are zero, and the zero-clocking constraints are met, this sequence of movements may be performed in reverse order. This procedure can be used to move all flip-flops back to their initial locations, using Theorem 4.2 to keep track of the altered clock skews at each flip-flop. The optimal retiming is equivalent to applying the set of skews thus obtained to the flip-flops in the circuit.

Note that the optimal clock period provided by the clock skew optimization procedure must, by definition, be no greater than the clock period for the set of clock skews thus obtained. Any differences arise due to the fact that clock skew optimization is a continuous optimization, while retiming is a discrete optimization. This argument leads to the following corollary:

Corollary 2 : The clock period obtained by an optimal retiming can be achieved in ASTRA via clock skew optimization. The clock period provided by the clock skew optimization procedure is less than or equal to that provided by the method of retiming.

Minperiod Retiming by ASTRA. The relationship between skew and retiming motivates the following two-phase solution to the retiming problem:

Phase A: The clock skew optimization problem is solved to find the optimal value of the skew at each FF, with the objective of minimizing the clock period, or to satisfy a given (feasible) clock period, using the procedure described in Chapter 3.

Phase B: The skew solution is translated to retiming and some FF's are relocated across gates in an attempt to set the values of all skews to be as close to zero as possible. Flip-flops with positive skews are moved opposite to the direction of signal propagation (i.e., in the backward direction), and those with negative skews are moved in the direction of signal propagation (i.e., in the forward direction) to reduce the magnitude of their skews.

The skew values obtained in Phase A are reduced in Phase B by applying retiming transformations using Theorem 4.2. This procedure relocates the FF's with nonzero skews across logic gates, while maintaining the optimal clock period previously found.

A gate can be retimed in the forward [backward] direction if it has FF's on each of its inputs [outputs], by removing one FF from each input [output] and adding one FF to each of its outputs [inputs].

The algorithm maintains two sets, one for the gates that are to be forward-retimed, and one for the gates that are to be backward-retimed. The forward retiming set, F, is initialized to contain all gates that have at least one FF on each of their inputs. Similarly the backward retiming set, B, is initialized with gates that have at least one FF on each their outputs. These sets are then processed by taking a gate from the set and retiming it if the skew on the FF's can be reduced by this retiming. After each such move, the sets F and B are updated. The pseudo code for this is given below as the function `RETIME` below:

```
Algorithm RETIME
{
  F = {v|v ∈ V and w(e_uv) ≥ 1 ∀u ∈ FI(v)} /* initialize F */
  B = {v|v ∈ V and w(e_vu) ≥ 1 ∀u ∈ FO(v)} /* initialize B */
  while(∃ u ∈ F) do FORWARD_RETIME(u, F);
  while(∃ u ∈ B) do BACKWARD_RETIME(u, B);
}
```

The functions `FORWARD_RETIME(gate,set)` and `BACKWARD_RETIME(gate,set)` retime the gate if needed, and update the respective sets. The corresponding pseudo code is listed below.

```
Algorithm FORWARD_RETIME(v, F)
{
  F ← F − v; /* remove gate v from F */
  x_i = maximum skew over all FF's at the fanins of gate v;
  x'_i = x_i + d(v);
  if (|x'_i| < |x_i|) do {
```

```
    /* retime gate v */
    for ∀u ∈ FI(v) do {w(e_uv) ← w(e_uv) − 1};
        /* delete an FF from each input */
    for ∀u ∈ FO(v) do {w(e_vu) ← w(e_vu) + 1};
        /* add an FF with skew = X_i' on all outputs */
    for ∀u ∈ FO(v) do
        if (w(e_wu) ≥ 1 ∀w ∈ FI(u)) do F ← F ∪ u; /* update F */
  }
  else
    Set skews of all FF's at the fanins of gate v to x_i.
}

Algorithm BACKWARD_RETIME(v, B)
{
  B ← B − v; /* remove gate v from B */
  x_i = minimum skew over all FF's at the fanouts of gate v;
  x_i' = x_i − d(v);
  if (|x_i'| < |x_i|) do {
    /* retime gate v */
    for ∀u ∈ FO(v) do {w(e_vu) ← w(e_vu) − 1};
        /* delete an FF from each output */
    for ∀u ∈ FI(v) do {w(e_uv) ← w(e_uv) + 1};
        /* add an FF with skew = X_i' on all inputs */
    for ∀u ∈ FI(v) do
        if (w(e_uw) ≥ 1 ∀w ∈ FO(u)) do B ← B ∪ u; /* update B */
  }
  else
    Set skews of all FF's at the fanouts of gate v to x_i.
}
```

As stated in Theorem 4.2, for forward retiming on a gate v, the effective skew before retiming, x_i, is given by the maximum skew at its inputs, while the effective skew after retiming, $x_i' = x_i + d(v)$. For backward retiming, the effective skew before retiming, x_i, is given by the minimum skew at its outputs, and the skew after retiming, $x_i' = x_i - d(u)$. In either case, the gate is retimed only if magnitude of the effective skew after retiming, x_i', is less than the magnitude of the effective skew before retiming, x_i. The gate is then forward [backward] retimed by removing an FF from each of its inputs [outputs] and adding an FF with skew X_i' to all its outputs [inputs]. After the forward [backward] retiming a gate v, the list of gates have at least one FF on each of its fanouts [fanins] is updated.

Retiming an FF forward across a gate affects the edge weights on only its own fanouts, and not the edge weights on fanouts of any other gate. Therefore, forward retiming a gate cannot enable the backward retiming of any other gate that could not be previously retimed in the backward direction. Moreover,

since a gate is retimed only if the effective skew magnitude is reduced by this move, a gate cannot be backward-retimed after it has been forward retimed once because such a backward retiming would increase the skew magnitude. Therefore, forward retimings have no effect on backward retimings and both types of moves can be carried out independently.

Although this is not shown in the pseudocode, in the final step, each FF whose skew remains larger than $\frac{d}{2}$, where d is the maximum delay of all gates that the FF fans out to, is set to zero skew. This result works since these FF's never became candidates for relocation during the process, since the maximum effective skew at their inputs turned out to be too large for them to be retimed. Therefore, their skew can be set to zero.

The following result is true by construction, since the retiming procedure is stopped at the point where the skew magnitude can no longer be improved by retiming.

Theorem 4.3 *At the end of the retiming procedure in Phase B, the skew at each FF is no more than half a gate delay.*

Theorem 4.4 *If, at the end of the retiming procedure, all skews are set to zero, then the optimal clock period for this circuit is no more than $P_s + d_{max}$, where P_s is the optimal clock period found in Phase A, and d_{max} is the maximum delay of any gate in the circuit.*

Proof: At the end of the retiming procedure, relation (4.8) can be used to infer that the long path constraints will be satisfied if the clock period is modified to:

$$\max(P_s + x_j - x_i) \tag{4.11}$$

where x_j and x_i are the skews of any pair of FF's such that there is a purely combinational path from i to j. By Theorem 4.3, since $|x_k| < d_{max}/2 \; \forall \; k$, the clock period required to ensure that all double-clocking constraints are satisfied is no more than $P_s + d_{max}$. Note also that since any period achievable by retiming is also achievable using skews, and final period can be no better than P, which is the optimal period using skew optimization. □

After Phase B, any skews that could not be set exactly to zero are forced to zero. This could cause the clock period to increase from Phase A; however, it is shown that this increase will be no greater than the maximum gate delay. Note, however, that this is not necessarily suboptimal since the minimum clock period using skews may not be achievable using retiming, since retiming allows cycle-borrowing only in discrete amounts (corresponding to gate delays), while skew is a continuous cycle-borrowing optimization [Fis90].

Experimental Results. Table 4.1 presents experimental results on minperiod retiming using ASTRA. For each circuit, the number of gates $|G|$, the optimal clock period P, and the number of FF's in the optimal circuit $|FF|$, are shown. Also presented are the CPU time (in seconds) in terms of the time

Table 4.1. Results of Minperiod Retiming using ASTRA

Circuit	$\|G\|$	P	$\|FF\|$	T_a	T_b	T_t
s13207.1	8004	51.00	629	0.34	0.20	0.54
s15850.1	9843	63.00	565	0.97	0.24	1.21
s35932	16420	27.00	1729	0.44	0.49	0.93
s38584.1	19595	48.00	1428	1.15	0.66	1.81
s38417	21504	32.00	1619	13.22	0.50	13.72
myex1	35711	35.00	3912	2.31	1.21	3.52
myex2	40993	35.00	7837	3.58	1.69	5.27
myex3	50532	47.00	11856	102.00	2.99	104.99

spent in Phase A (T_a), the time spent in Phase B (T_b) and the total execution time (T_t). All results have been generated on a Sun UltraSparc 1/170 workstation with 128MB of memory. It is seen that a 50,000 gate circuit can be retimed within two minutes.

4.4.3 Retiming for a Target Period

Retiming a circuit for a given target clock period is a special case of the minperiod retiming problem. Given a circuit and a clock period P, if the given clock schedule is feasible, then the method should return a retimed circuit that is correctly clocked, and if the clock schedule is not feasible, this should be indicated by the method. In the Leiserson-Saxe method, a specified-period retiming is obtained by simply running the algorithm FEAS with the target clock period. In the ASTRA approach for this problem, the binary search in Phase A is not performed, but for the target clock period P only, the constraint graph is constructed as before, and the Bellman-Ford algorithm is applied to obtain the set of required skews. If the Bellman-Ford algorithm detects a positive cycle, then the clock period is not feasible, and is reported as such; otherwise Phase B is performed.

A retiming for a given clock period is, in general, not unique, and different retimed circuits can be obtained, all of which satisfy the target clock period. This may be understood through the presence of slacks in the graph at the end of the Bellman-Ford procedure, which may be used to alter the skew values within a specified range without altering the feasibility. In minperiod retiming, the objective is to find one of these possible solutions efficiently. Therefore, it makes sense to minimize the number of retiming moves. This is achieved by initializing the skews to 0 in the Bellman-Ford algorithm, and using the slacks to minimize the number of moves.

The ALAP and ASAP Retimings. Out of the set of all possible specified-period retimings, two are of particular interest. The retiming that moves all registers as far as possible against the direction of signal propagation is called an "as soon as possible" (ASAP) retiming. Similarly, the retiming that moves all the registers as far as possible in the direction of signal propagation is referred to as the "as late as possible" (ALAP) retiming. Both ASAP and ALAP retiming assume that no register is moved across the host node.

In this section we will concentrate on explaining the ASAP and ALAP retimings in the context of edge-triggered circuits. However, the reader should remain aware that the same basic principle applies to level-clocked retimings, even though we do not address the issue in detail here.

The ASAP and the ALAP locations can be seen as the extreme locations of the locations of the registers in the circuit for the specified clock period, and are unique. Their importance is in the fact that they may be utilized to make the procedure of minarea retiming more efficient, as explained in Section 5.2.

For the example circuit in Figure 4.11, with unit delay inverters and edge-triggered FF's, the ASAP and ALAP retimings for a target clock period of 3 units are shown in Figure 4.12 and Figure 4.13, respectively.

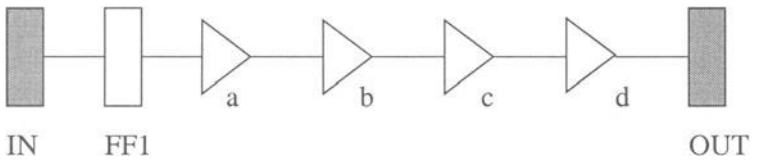


Figure 4.11. An example circuit.

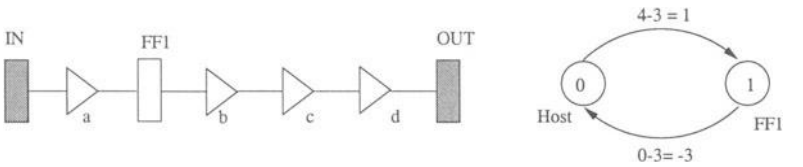


Figure 4.12. ASAP retiming of the example circuit, with the ASAP constraint graph.

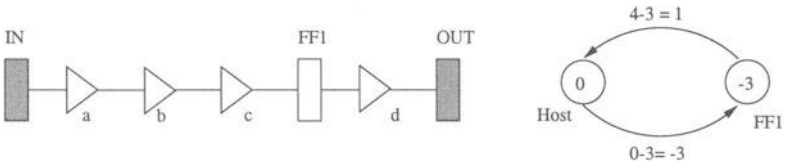


Figure 4.13. ALAP retiming of the example circuit, with the ALAP constraint graph.

In ASAP retiming, the objective is to move the FF's as far as possible in the backward direction. Since Phase B of ASTRA moves FF's with positive skews in the backward direction, an ASAP retiming would aim to obtain the

maximum possible skew value for each register in Phase A. Similarly, it can be argued that FF's with negative skews would be at their most backward location if the skews were set to be at their maximum (least negative) value. Therefore, in either case, an ASAP retiming corresponds to the maximum allowable skew values; symmetrically, an ALAP retiming corresponds to the minimum possible (least positive and most negative) skew values. The ASAP skews are obtained by running the Bellman-Ford algorithm on the transpose [CLR90] of the original constraint graph, i.e., a graph with the same vertex set as the original graph, but with the edge directions reversed. In this transpose graph, an edge from u to v exists if there is a combinational path from FF v to FF u; the weight of this edge is $\overline{d}(v,u) - P$. The initial skew values for the Bellman-Ford algorithm are set to $-\infty$ for all FF's, except the host which is initialized to 0. Since all of the edge directions have been reversed, the longest path values obtained for each node in the graph must be made to undergo a sign reversal to obtain the correct skew values for the corresponding FF's. The skew values so obtained are the maximum possible for the specified clock period, and no skew can be increased further without violating the clock period. For example, the transposed Phase A constraint graph for ASAP retiming of the circuit in Figure 4.11, is shown in Figure 4.12. The longest path to the node corresponding to FF1 is +1 unit, and hence, the corresponding skew on FF1 of -1 unit is obtained by reversing the sign.

Performing Phase B with these skew values results in the ASAP retiming. For ASAP locations, the available slack is used to avoid moving an FF in the direction of signal flow. The procedure for finding the ASAP (and ALAP) retiming proceeds along the same lines as in Section 4.4.2, with a few variations described below.

To obtain the ASAP locations for the retimed FF's, it is necessary to push the FF's as far as possible in a backward direction. Therefore, each FF with positive skew is moved as far as possible in the backward direction, and each FF with negative skew is moved as little as possible in the forward direction. Therefore,

(1) for an FF with positive skew x that is being moved across a single-fanout gate p in the backward direction, the skew value after the relocation at input i of p is set to $x - \text{delay}(p)$. If this value is nonpositive, then the ASAP location has been found. For gates with multiple fanouts, $x = \min_{\text{all outputs}}(x_i)$, where x_i is the skew of the FF at the ith output, as shown in Figure 4.14(a).

(2) for an FF with negative skew x that is being moved across a single-fanin gate p in the direction of signal propagation, the skew value after the relocation at output i of p is set to $x + \text{delay}(p) + \text{slack}(i)$, where slack($i$) is the slack associated with the output i. This slack is defined as the amount by which the delay at output i may be increased before it becomes the critical output of p; by definition, the critical output has zero slack. If the new skew is non-negative, then the ASAP location has been found. For gates with multiple

fanins, $x = \max_{\text{all inputs}}(x_i)$, where x_i is the effective skew of the FF at the ith output, as shown in Figure 4.14(b).

Applying this procedure to circuit in Figure 4.11 requires us to move FF1, which has a skew of -1 unit, across the unit delay inverter a to the location shown in Figure 4.12. At this point, the skew on FF1 is down to zero, and hence, the ASAP location has been reached. As is shown in this example if the initial circuit does not satisfy the target clock period, then ASAP retiming may require FF's to move forward, and these forward moves will be required by *all* retimings satisfying the target clock period.

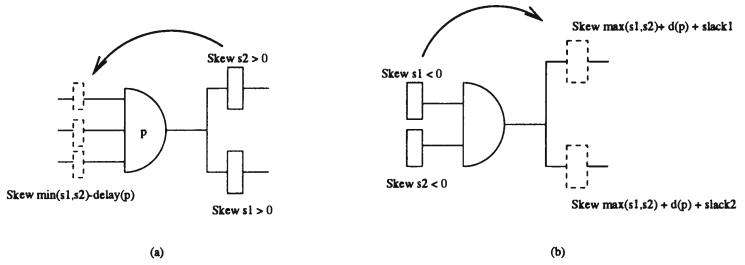


Figure 4.14. Effective skews at FF's after ASAP retiming across a gate.

Since the objective in ALAP retiming is to move the FF's as far as possible in the forward direction, Phase A of ASTRA is modified to yield the minimum (most negative and least positive) skew values. All skew values are initialized to $-\infty$, except the host which is set to zero, before applying the Bellman-Ford algorithm to find the longest path in the constraint graph. In Phase B of ALAP retiming, each FF is moved as far as possible in the forward direction, using slacks if any to minimize any FF movements in the backward direction.

The Phase A constraint graph for ALAP retiming of the circuit in Figure 4.11 is shown in Figure 4.13. The longest path to the node corresponding to FF1 is +3 units, and hence, the skew on FF1 is 3 units. Since FF's with positive skews are moved as far forward as possible for ALAP retiming, FF1 moves across gates a, b, and c until its skew is brought down to zero and the ALAP location has been achieved.

4.4.4 Continuous Retiming

A related technique to the ASTRA algorithm is the procedure of continuous retiming (c-retiming) [Pan97]. The motivation for c-retiming is its potential use in studying synthesis and optimization problems that involve circuit modifications in conjunction with retiming, and an application of c-retiming to the tree mapping problem [De 94] is presented in [Pan97]. Unlike retiming, this method assigns a real number to each gate, representing the fractional number of FF stages to be moved across the gate. Rounding off this fractional number of FF's will result in a conventional retiming, with the resulting clock period guaranteed to be within the largest gate delay of the optimal clock period. This

provides an efficient way for minperiod retiming, since c-retiming can be solved as a single-source longest path problem. In c-retiming, the edge weights ($w_s(e)$) are also real numbers and denote the number of fractional FF's present on an edge e. The clock period of the c-retimed circuit is P if the final continuous retimed weight on each edge is no smaller than $\frac{d(v)}{P}$.

4.5 LEVEL-CLOCKED CIRCUITS

For edge-triggered circuits, the delays through all combinational logic paths must be less than the clock period, with allowances for the setup time. Therefore, timing constraints need only be enforced between FF's connected by a purely combinational path. For level-clocked circuits, the delay through a combinational logic path may be longer than one clock cycle, as long as it is compensated by shorter path delays in subsequent cycles. To ensure that the extra delay is compensated, timing constraints must be enforced from each latch to every other latch reachable from it, possibly through a path that traverses multiple latches. This greatly increases the complexity of the problem for practical circuits, as was illustrated by the example of an N-stage acyclic pipeline in Section 2.5.

4.5.1 Variation of the Critical Path with the Clock Period

As in the case of edge-triggered circuits, traditional methods [PR93b, LE94] solve the minperiod retiming problem for level-clocked circuits by performing a binary search over all possible clock periods. At each step of this binary search, the feasibility of achieving the clock period by retiming is checked by solving a single source shortest path problem, using the Bellman-Ford algorithm on a constraint graph. For a circuit with $|G|$ gates, this constraint graph consists of $|G|$ vertices and edges between every pair of vertices, constructed by solving an all-pairs shortest path problem on the original circuit graph. This graph must be reconstructed for every binary search point, because will be shown shortly, unlike in the case of edge-triggered circuits, critical paths in level-clocked circuits can be different for different clock periods. Therefore, the methods in [PR93b, LE94] have $O(|G|^2)$ memory requirements and high (although polynomial) time complexity.

In an edge-triggered circuit, the critical path is independent of the clock period, and simply corresponds to the output with the largest delay, except in the uninteresting case when the clock period is satisfied. In a level-clocked circuit, however, critical paths are more difficult to identify since they may vary from one clock period value to another. This was illustrated in [LE94] through the following example. Figure 4.15 shows a situation where three paths, p_1, p_2, and p_3, connect the vertex u to the vertex v; the delay of each node is written within its corresponding vertex. The description of each path is as follows:

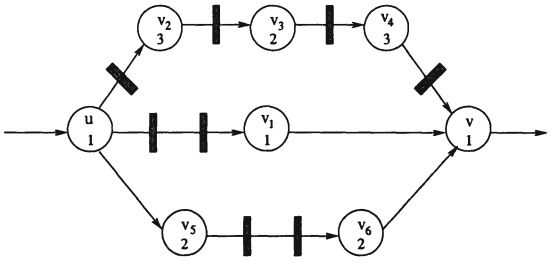


Figure 4.15. Example illustrating the change in the critical path with the clock period [LE94] ©1994 IEEE.

Path	Vertices	$w(p)$	Path delay
p_1	$u \Rightarrow v_2 \Rightarrow v_3 \Rightarrow v_4 \Rightarrow v$	4	10
p_2	$u \Rightarrow v_1 \Rightarrow v$	2	3
p_3	$u \Rightarrow v_5 \Rightarrow v$	2	6

where $w(p)$ denotes the number of registers that currently lie on the path.

If a path from u to v has w registers on it, then for a symmetric two-phase clock with an active period of T_ϕ, the amount of time available for data to travel from u to v is $(w + 2) \cdot T_\phi$ time units, corresponding to the total active time available from the input register of node u to the output register of node v. This leads to the following statements:

- When a symmetric two-phase clock with a period of 3 units (and therefore, an active time of 1.5 units) is applied, we require that $w_r(p_1) = 5$ and $w_r(p_3) = 2$. Therefore, the path p_1 is critical since satisfying the constraint on p_3 in the given circuit would violate the constraint on p_1.
- When the period of the same clock is changed to 5 units, the requirements are altered to $w_r(p_1) = 2$ and $w_r(p_3) = 1$; in this case, satisfying the former would lead to a violation of the latter.
- It is easily verified that the crossover point for the critical paths occurs at a clock period of 4 units, where both are critical.

Note that the path p_2 is always subservient to path p_3 since it has the same number of registers, but a smaller delay, and can, hence, never be critical.

4.5.2 *Retiming of Single Phase Level-clocked Circuits*

In [SBS91] an MILP formulation for retiming single-phase level-clocked circuits is presented. Constraints for correct clocking of single-phase circuits are laid down, which were essentially similar to the SMO constraints. A new functionality constraint was also introduced in this work, which maintains temporal equivalence between the initial and the retimed circuit. The constraint identifies a set of fundamental cycles in the directed graph corresponding to this

problem, and ensures that the sum of latches on each of these is maintained during retiming. The property of the set of fundamental cycles is that all other cycles can be expressed as a composition of these fundamental cycles. The essential idea of the approach is to build a spanning tree on the directed graph. If the graph has m edges and n vertices, then there are $m - n + 1$ edges that are excluded from the spanning tree. It is proved in [SSBS92] that each of these edges corresponds to a linearly independent cycle by showing that the cycle-edge incidence matrix has rank $m - n + 1$.

4.5.3 *The TIM Approach*

TIM [Pap93, PR93b] presents polynomial time algorithms for retiming and clock tuning of two-phase level clocked circuits. Since the notation used in the work differs slightly from the SMO model, we will pause for a few definitions. A clocking scheme is defined as $\pi = \langle \phi_0, \gamma_0, \phi_1, \gamma_1 \rangle$, where P is the clock period, ϕ_i the duty cycle of phase i and γ_i is the time gap between the instant where the clock phase i goes low and the clock phase $i(i+1) \bmod k$ goes high. Each vertex, v, corresponds to a gate, and is associated with a phase, $\chi(v) \in \{0, 1\}$, i.e., $\phi_{\chi(v)}$ clocks the last latch on any edge that ends at v.

The constraint for proper clocking on a path $p : u \rightsquigarrow v$ is given by the following relations:
If $\chi(u) \neq \chi(v)$,

$$\begin{aligned} d(p) \leq\ & P\left(\frac{1 + w(p)}{2}\right) + \phi_{\chi(u)} \\ & + P\lfloor \frac{r(v)}{2} \rfloor + (r(v) \bmod 2)\left(\gamma_{\chi(u)} + \phi_{1-\chi(u)}\right) \\ & - P\lfloor \frac{r(u)}{2} \rfloor - (r(u) \bmod 2)\left(\gamma_{\chi(u)} + \phi_{\chi(u)}\right) \end{aligned} \tag{4.12}$$

If $\chi(u) = \chi(v)$,

$$\begin{aligned} d(p) \leq\ & P\left(\frac{2 + w(p)}{2}\right) - \gamma_{1-\chi(u)} \\ & + P\lfloor \frac{r(v)}{2} \rfloor + (r(v) \bmod 2)\left(\gamma_{1-\chi(u)} + \phi_{\chi(u)}\right) \\ & - P\lfloor \frac{r(u)}{2} \rfloor - (r(u) \bmod 2)\left(\gamma_{\chi(u)} + \phi_{\chi(u)}\right) \end{aligned} \tag{4.13}$$

When $\chi(u) \neq \chi(v)$, $1 - \chi(u) = \chi(v)$ and the relation (4.12) enforces the constraint that the delay on any path must be less than the summation of the following terms (the interpretation of relation (4.13) is analogous):

- The first line on the right hand side is the timing allowance on the path p before retiming.
- The second line is the change in the timing allowance of the path p due to the latches that where moved either on or off p through gate v during the

retiming process. The first term ($P\lfloor\frac{r(v)}{2}\rfloor$) is the number of complete clock periods added to the timing allowance, and the second term accounts for fractional periods added. If $r(v)$ is even, no fractional periods are added and the second term is zero ($r(v) \bmod 2 = 0$). If $r(v)$ is positive, latches are shifted on to path p and the timing allowance is increased; if it is negative, latches are being shifted off from path p and the timing allowance is decreased.

If $r(v)$ is odd, the fractional change must also be considered. For a positive value of $r(v)$, the last latch moved on p is of phase $1 - \chi(v)$. Therefore, the latch on the boundary of p (which includes gate v) will be of phase $1-(1-\chi(v)) = \chi(v)$ which is same as $1-\chi(u)$. Thus, the fractional period added by this last latch is $\gamma_{\chi(u)} + \phi_{1-\chi(u)}$. If $r(v)$ is negative, then the last latch moved off p by forward retiming across v is of phase $\chi(v)$, and the quantity $\gamma_{1-\chi(v)} + \phi_{\chi(v)} = \gamma_{\chi(u)} + \phi_{1-\chi(u)}$ must be subtracted from the timing allowance of path p.

- The third line is the change in the timing allowance of the path p due to latches being moved off or on p through gate u, and is derived in a similar manner to the second line.

For a more formal mathematical proof, the reader is referred to [Pap93].

These constraints in relations (4.12) and (4.13), together with the nonnegativity constraint, $r(v) + w(e_{uv}) \geq r(u)$, form an integer monotonic program that can be solved using graph-based algorithms.

A similar method for retiming level-clocked circuits was independently presented in [LE91, LE92, LE94], and was based on the SMO formulation of timing constraints.

4.5.4 *Relation Between Retiming and Skew for Level-clocked Circuits*

The work in [MS96] extends the ASTRA algorithm to level-clocked circuits. To derive timing constraints in the presence of skews, the SMO model of Section 2.5.2 is augmented with new notation. A skew, x_i, is associated with every latch $i \in \Psi$, where Ψ is the set of all latches in the circuit. It is worth pointing out at this juncture that the skew values, in the context of this algorithm, are not physical skews to be applied to the final circuit, but conceptual ideas that will eventually lead to achieve a retiming solution. However, the equations are valid for any multiphase clocking scheme in the presence of skews. No restrictions are placed on the value of x_i, i.e. $-\infty \leq x_i \leq \infty$.

We define a *latch shift* operator $L_{i,j}$, shown in Figure 4.16, much like the phase shift operator in the SMO formulation. This operator converts time from the local time zone of latch i to the local time zone of latch j, taking into account their skews. It is defined as

$$L_{i,j} = \begin{cases} (x_j + e_{p(j)}) - (x_i + e_{p(i)}) & \text{for } i < j \\ T_\Phi + (x_j + e_{p(j)}) - (x_i + e_{p(i)}) & \text{for } i \geq j \end{cases}$$

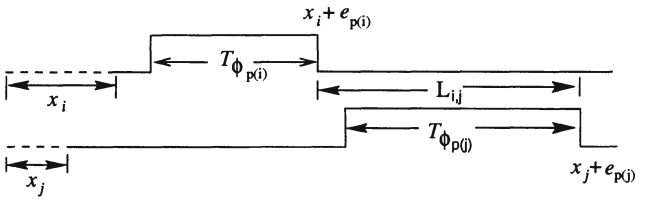


Figure 4.16. The latch shift operator.

which can be rewritten in terms of the phase shift operator as

$$L_{i,j} = (x_j - x_i) + E_{p(i),p(j)} \tag{4.14}$$

In presence of skews at latches, the timing constraints in relation (2.15) and (2.18), must be modified by using the latch shift operator instead of the phase shift operator; only long path constraints are considered here. Thus, the timing constraints for a level clocked circuit to be properly clocked by a clock schedule, Φ, in presence of skews are

$$\begin{aligned} D_i + d(i,j) - L_{i,j} &\leq D_j \quad \forall\, i \rightsquigarrow j \mid i,j \in \Psi \\ T_\Phi - T_{p(i)} \leq D_i &\leq T_\Phi \quad \forall\, i \in \Psi \end{aligned}$$

For simplicity, this discussion ignores the clock-to-Q delays and setup times, though these may easily be inserted into these equations. These timing constraints can be rewritten as

$$\begin{aligned} (x_i + D_i) + d(i,j) - E_{p(i),p(j)} &\leq (x_j + D_j) \quad \forall\, i \rightsquigarrow j \mid i,j \in \Psi \\ T_\Phi - T_{p(i)} &\leq D_i \leq T_\Phi \quad \forall\, i \in \Psi \\ -\infty &\leq x_i \leq \infty \quad \forall\, i \in \Psi \end{aligned}$$

To make the discussion simpler, we subtract T_Φ from both sides of the first relation, and substitute

$$X_i = (x_i + D_i - T_\Phi) \tag{4.15}$$

The quantity X_i is referred to as the *Global Departure Time* (GDT). Therefore, we have

$$\begin{aligned} X_i + d(i,j) - E_{p(i),P(j)} &\leq X_j \quad \forall\, i \rightsquigarrow j \mid i,j \in \Psi \\ -\infty \leq X_i &\leq \infty \quad \forall\, i \in \Psi \end{aligned}$$

These can be rewritten as the following set of difference constraints:

$$\begin{aligned} X_i - X_j &\leq E_{p(i),p(j)} - d(i,j) \quad \forall\, i \rightsquigarrow j \mid i,j \in \Psi \\ -\infty \leq X_i &\leq \infty \quad \forall\, i \in \Psi \end{aligned} \tag{4.16}$$

The difference constraint between GDT values of two latches given in relation (4.16) is similar to the difference constraints between skews at FF's in

relation (4.8). This suggests a relation between retiming and GDT values of level-sensitive latches, similar to the retiming-skew relationship for edge triggered FF's. The following theorem is similar to Theorem 4.2.

Theorem 4.5
(a) *Retiming transformations may be used to move latches from all of the n inputs of any combinational block to all of its outputs. The equivalent GDT of the relocated latch at output j, considering long path constraints only, is given by*

$$X_j = \max_{1 \le i \le n} (X_i + \overline{d}(i,j))$$

where the $X_i, 1 \le i \le n$, *are the GDT's at the input latches,* X_j *is the equivalent GDT at output* j, *and* $\overline{d}(i,j)$ *is the worst-case delay of any path from* i *to* j.
(b) *Similarly, latches may be moved from all of the m outputs of any combinational block to all of its inputs, and the equivalent GDT at input k, considering long path constraints only, is given by*

$$X_k = \min_{1 \le j \le m} (X_j - \overline{d}(k,j)) \qquad (4.17)$$

where the $X_j, 1 \le j \le m$, *are the GDT's at the input latches,* X_k *is the equivalent GDT at input* k, *and* $\overline{d}(k,j)$ *is the worst-case delay of any path from* k *to* j.

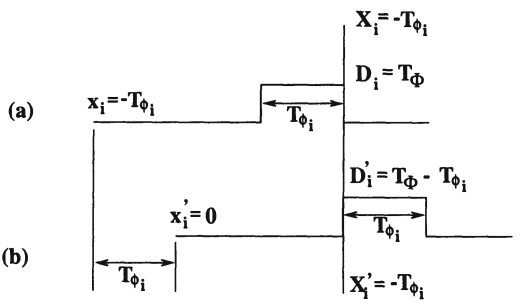


Figure 4.17. The ability of a latch to absorb some skew.

The physical meaning of the GDT is as follows. If arbitrary skews could be applied at latches, the skew, x_i, of a latch could be adjusted so as to force $D_i = T_\Phi$, which is same as a negative edge triggered FF. Since $X_i = x_i + D_i - T_\Phi$, setting $D_i = T_\Phi$ gives $x_i = X_i$. Hence, we can conceive of the GDT, X_i, for a latch to be similar to the skew for an FF.

Note that in reality, we are not compelled to set $D_i = T_\Phi$, and that D_i can be reduced by as much as T_{ϕ_i}, if x_i is increased by the same amount while keeping X_i constant. Since only GDT's, X_i, appear in the timing constraint of relation (4.16), keeping them constant keeps the clock period constant. As

an illustration of this idea, consider Figure 4.17(a) where $x_i = -T_{\phi_i}$ and $D_i = T_\Phi$; this implies that $X_i = -T_{\phi_i}$. The skew here may be increased to zero ($x_i' = 0$), without changing the GDT, as shown in Figure 4.17(b), by reducing the departure time by the same amount $D_i' = T_\Phi - T_{\phi_i}$, thereby leaving the GDT unchanged ($X_i' = X_i = -T_{\phi_i}$). Therefore, a skew of up to $-T_{\phi_i}$ can be absorbed in the D_i without violating the long path constraint. Thus, any GDT value between $-T_{\phi_i}$ and 0 is permissible, and this range will be referred to as the allowable range. If different phases have different active intervals, then the allowable GDT range of a latch will depend on its phase. Therefore, in this model, level-sensitive latches can be conceptualized as FF's that have the capacity to absorb some skew.

At this time, we also note the relation between the GDT, X_i, of a latch i and the corresponding minimum magnitude skew, x_i:

$$x_i = \begin{cases} X_i & \text{if } X_i \geq 0 \\ 0 & \text{if } -T_{\phi_i} \leq X_i \leq 0 \\ X_i + T_{\phi_i} & \text{if } -T_{\phi_i} > X_i \end{cases} \tag{4.18}$$

4.5.5 *Minimum Period Retiming Algorithm*

The relationship between GDT and retiming presented in Section 4.5.4 can be utilized to map the problem of retiming level-clocked circuits for minimum period to the simpler problem of retiming edge-triggered circuits for minimum period as solved in Section 4.4.2. This mapping motivates the following two-phase method of retiming for minimum clock period under a specified clocking scheme.

Phase A: Find the minimum clock period and a set of GDT values that will achieve this period.

Phase B: Relocate latches across gates to get all the GDT values to be within allowable range.

The advantage of this method is that it constructs a small and sparse graph only once, unlike the traditional methods [PR93b, LE94] that construct multiple large and dense graphs. In Phase A, the skew optimization problem is solved as though the latch were a negative edge-triggered FF. In Phase B, fast local transforms are used to obtain the retiming solution. Although the latching edge in Phase A is fixed to be the negative edge, this is not a strict restriction; it is removed in Phase B when the flexibility in the GDT value is utilized to seek a zero-skew solution. This two-phase method can be used to retime large level-clocked circuits very efficiently.

As in Section 4.4.2, it must be noted that since gate delays take on discrete values, it cannot be guaranteed that the GDT at every latch can be reduced to be within the allowable range through retiming operations. After the GDT values have been reduced as much as possible, the retimed circuit may be implemented either by applying the requisite skews at a latch to get the optimal clock period achievable by skew optimization, or by setting all skews to zero

to get a clock period that is, as will be shown shortly, no more than a fixed bound above the optimal period with skews. As in the case of edge-triggered circuits, this is not necessarily suboptimal since the minimum clock period using skews may not be achievable by retiming alone, since retiming allows cycle-borrowing only in discrete amounts (corresponding to gate delays), while skew is a continuous cycle-borrowing optimization. As will be shown later in this section, if the maximum gate delay is less than the least T_{ϕ_i}, the optimal skew optimization period can always be achieved by retiming.

We first describe the two phases of minperiod retiming, followed by the special case of retiming a circuit for a specified clock period. We then present the bound on the difference between the optimal skew optimization period and the clock period obtained by our method.

Phase A: Clock Period Optimization

The problem of minimizing the clock period, T_Φ, for a specified clocking scheme can be represented as the following linear program:

$$\begin{aligned} &\text{minimize} && T_\Phi \\ &\text{subject to} && X_i - X_j \leq E_{p(i),p(j)} - d(i,j) \quad i \rightsquigarrow j \mid i,j \in \Psi \end{aligned} \tag{4.19}$$

The solution to the above linear program yields the minimum clock period and the corresponding GDT's. Since for a specified clock schedule that has a fixed T_Φ, $E_{p(i),p(j)}$ is constant, the constraint matrix reduces to a system of difference constraints. Thus, the LP is solved as for edge-triggered circuits in Section 3.4.3.

Phase B: GDT Minimization

In Phase B, the GDT values obtained in Phase A are reduced by applying retiming transformations using Theorem 4.5. This procedure relocates the latches with nonzero GDT's across logic gates, while maintaining the optimal clock period previously found. Because of the freedom provided to D_i by the active interval of clock phase $p(i)$, which allows D_i to be set to any value between $T_\Phi - T_{\phi_{p(i)}}$ and T_Φ, $x_i = 0$ can be achieved if $-T_{\phi_{p(i)}} \leq X_i \leq 0$. If x_i cannot be set to zero, then X_i is brought as close to 0 or $-T_{\phi_{p(i)}}$ as possible so as to minimize the magnitude of the final skew x_i.

The method for FF relocation presented in Section 4.4.2 is modified to work for latches. The decision to move a latch is now based on comparing the minimum magnitude skew required before and after retiming, as given by equation (4.18), rather than by comparing the GDT's (which would have been the natural extension of the edge-triggered procedure). The following bound can be placed on the unresolved skew and the final clock period obtained at the end of Phase B.

Theorem 4.6 *At the end of the retiming procedure in Phase B, the magnitude of skew at each latch i, is no more than*

$$\gamma_{p(i)} = \max\left(0, \frac{M - T_{\phi_{p(i)}}}{2}\right)$$

where M is the maximum delay of any gate in the circuit.

Proof: Two cases must be considered:

Case A : $M \leq T_{\phi_{p(i)}}$ If the maximum gate delay is less than the active duration of the clock, it must be proved that at the end of Phase B, all latches will have zero skew. This is proved by contradiction, assuming that a latch i has nonzero skew x_i at the end of Phase B. There are two subcases.

Subcase 1: $x_i > 0$ In this case, the GDT of latch i is $X_i = x_i$. The new GDT of the latch after it is moved left across a gate of delay d_1 is given by $X_i' = X_i - d_1$. Since $d_1 \leq M \leq T_{\phi_{p(i)}}$, we have $X_i' \geq -T_{\phi_{p(i)}}$; therefore, the effective skew x_i' after this possible move is either zero (if $T_{\phi_{p(i)}} \leq X_i' \leq 0$), or $|x_i'| < |x_i|$. In either case, the latch i can be moved left across the gate and have its skew reduced. This contradicts the assumption that Phase B is complete.

Subcase 2: $x_i < 0$ If $x_i < 0$ then the GDT of latch i is negative, i.e., $X_i = x_i - T_{\phi_{p(i)}}$, and the proof proceeds in a similar manner as subcase 1.

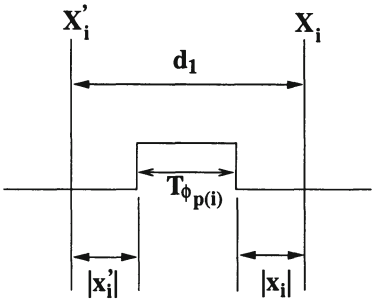


Figure 4.18. Worst-case situation for remaining skew.

Case B: $M > T_{\phi_{p(i)}}$ If the maximum gate delay is more than the active duration of the clock, we need to prove that at the end of Phase B, the skew magnitude for any latch i is less than $\gamma_{p(i)}$. Phase B is complete only when for every latch i, $|x_i'| \geq |x_i|$, where x_i is the current skew and x_i' is the skew after a possible move across a gate with delay d_1. As shown in Figure 4.18, the largest possible final skew magnitude corresponds to the situation when $|x_i| = |x_i'|$. In this case, we have $d_1 = 2 \cdot |x_i| + T_{\phi_{p(i)}}$, and hence, $|x_i| \leq \frac{d_1 - T_{\phi_{p(i)}}}{2}$. Since M is the maximum delay of any gate in the circuit, this implies that $|x_i| \leq \frac{M - T_{\phi_{p(i)}}}{2}$.

Theorem 4.7 *If, at the end of the retiming procedure, all skews in a k-phase circuit are set to zero, then the final clock period (P_r) satisfies the following*

condition

$$P_r \leq P_s + \sum_{i=1}^{k} \max(0, M - T_{\phi_i})$$

where P_s is the optimal clock period with skews found in Phase A, and M is the maximum delay of any gate in the circuit.

Proof : Each difference constraint for the optimal clock period (with skews) is of the form

$$X_i - X_j \leq E_{p(i),p(j)} - d(i,j).$$

Theorem 4.6 guarantees us that at the end of Phase B, $|X_i|$ and $|X_j|$ are within $\gamma_{p(i)}$ and $\gamma_{p(j)}$ of their optimal values respectively. Therefore, the actual value of $X_i - X_j$ after Phase B must lie within $(\gamma_{p(i)} + \gamma_{p(j)})$ of the required value of $X_i - X_j$ in Phase A. This implies that the inequality that defines the difference constraint can be maintained by increasing $E_{p(i),p(j)}$ by no more than $(\gamma_{p(i)} + \gamma_{p(j)})$. Since each $E_{i,j}$ increases by no more than $(\gamma_{p(i)} + \gamma_{p(i)})$, the clock period $T_\Phi = \sum_{i=1}^{k} E_{p(i),p(j)}$ will increase by no more than $\sum_{i=1}^{k}(2 \cdot \gamma_{p(i)}) = \sum_{i=1}^{k} \max(0, M - T_{\phi_i})$.

4.5.6 Experimental Results

The minimum period retiming algorithm for level-clocked circuits, described above, was implemented as a C program, ASTRA-L [MS96]. Experimental results from applying this program to several two-phase circuits are shown in Table 4.2. For each circuit, the number of gates $|G|$, the optimal clock period P, the number of FF's in the optimal circuit $|FF|$. Also presented are the CPU time (in seconds) in terms of the time spent in Phase A (T_a), the time spent in Phase B (T_b) and the total execution time (T_t). As before, the CPU times reported are for a Sun UltraSparc 1/170 workstation with 128 MB of RAM. The execution times are extremely fast and a 50,000 gate circuit is retimed in about seven minutes.

4.6 CONCLUDING REMARKS

The retiming problem was introduced in this chapter, and some basic formulations and algorithms presented by Leiserson and Saxe have been overviewed. Practical implementational issues related to algorithms for solving these problems have also been examined, both for edge-triggered circuits and for level-clocked circuits. The relationship between clock skew and retiming has been presented for both classes of circuits. Efficient algorithms exploiting this relationship, which can handle circuits with tens of thousands of gates in minutes on an everyday workstation, have been presented.

Table 4.2. Results of Minperiod Retiming of 2 Phase Circuits using ASTRA-L

Circuit	$\|G\|$	P	$\|FF\|$	T_a	T_b	T_t
s13207.1	8004	51.00	1273	0.36	0.27	0.63
s15850.1	9843	63.00	1639	1.01	0.58	1.59
s35932	16420	27.00	5207	6.38	1.36	7.74
s38584.1	19595	48.00	8889	1.19	1.36	2.55
s38417.	21504	31.50	4863	26.68	1.40	28.08
myex1	35711	35.00	11258	2.75	3.16	5.91
myex2	40993	35.00	16291	101.03	2.66	103.69
myex3	50532	46.75	22475	419.39	5.57	424.96

Notes

1. An alternative view of level-clocked latches is as flip-flops that naturally permit a skew range of up to the active range of the latch. If, for example, the rising edge of the clock is set to time zero and the active pulse width of the clock is w, then a level-clocked latch has the same effect as an edge-triggered latch with an allowable skew of 0 through $+w$.

2. Since y is a negative number, $d(v) - y$ will be positive, as expected.

3. The calculation of indices i here span the range $0, \cdots, k$, and in this discussion, $i - 1$ is calculated modulo k.

4. This result is easy to see on this simple example. The reader is referred to Chapter 3 for a general procedure for finding the optimal skews for a general circuit

5. In general, it is not possible to come up with an equivalent skew value that satisfies both long and short path constraints. For example, when we consider short path constraints, moving flip-flops from the input to the output requires that the new skew be

$$\min_{1 \le i \le n} (x_i + \underline{d}(i, j))$$

(using the same notation as Theorem 4.2(a)), which is incompatible with the requirement stated above except in the special case where all paths from i to j have the same delay. This is not an impediment here since the retiming problem as stated by Leiserson and Saxe in [LS91] considers long path constraints only. Note that Theorem 4.1 is valid for both long path and short path constraints, and can be considered to be a special case where the combinational subcircuit has only one path, where clearly the longest and shortest path delays through the gate are equal.